Dynamic Load Balancing for a Real-Time Analytics Dashboard: A Comparative Evaluation of Round-Robin, Least-Connections, and Latency-Aware Routing

Ana Maria Floristean, Razvan-Constantin Croitoru
Faculty of Computer Science
Alexandru Ioan Cuza University, Iaşi, Romania
anafloris23@gmail.com, croitorurazvan1@gmail.com

***Abstract*—Distributed systems that serve real-time analytics must spread incoming requests across multiple backend workers without creating hotspots or causing cascading failures. In this paper, we build a containerised testbed with three worker services – two fast workers and one intentionally degraded worker with a 4× slowdown factor – and a custom load balancer supporting three routing algorithms: static round-robin, least-connections, and a latency-aware strategy based on Exponentially Weighted Moving Average (EWMA) scoring. We compare these algorithms under four scenarios: constant load, traffic spikes, progressive stress to 300 virtual users, and worker failure injection. Our results show clear and meaningful differentiation between algorithms. Under constant moderate load (30 VUs), the latency-aware algorithm routes only 5% of traffic to the slow worker (vs. 33% for round-robin), yielding 30% lower average latency and 19% higher throughput. Under extreme stress (300 VUs), both adaptive algorithms sustain over 229 req/s with under 2% failures, while round-robin reaches only 173 req/s with 7.6% failures and drives all workers into an unhealthy state. In the failure-injection scenario, the latency-aware algorithm detects the crashed worker within one health-check interval and reroutes traffic with just 0.01% overall failures. These findings confirm that adaptive routing algorithms provide substantial benefits both under normal conditions and at saturation.**

I. Introduction

When you have a web application that gets a lot of traffic, you cannot run everything on a single server. You need multiple backend servers (we call them "workers") to share the load. The component that decides which worker gets each request is called a *load balancer*.

The simplest approach is round-robin: send the first request to worker 1, the second to worker 2, the third to worker 3, then cycle back. This works well when all workers are identical and equally fast. In reality, workers may have different hardware, suffer from garbage-collection pauses, or crash entirely.

Dynamic load balancing means the system watches real-time signals – response time, queue depth – and uses that information to make smarter routing decisions.

Our research question is: **Which load-balancing strategy provides the best trade-off between throughput, tail latency, and reliability in a distributed real-time analytics system under variable workload, heterogeneous workers, and failures?**

To answer this we built a containerised distributed system from scratch and ran controlled experiments comparing three algorithms. A key finding is that even with only three workers and a 4× performance gap, adaptive algorithms measurably outperform round-robin across all tested conditions.

II. Related Work

Load balancing is a well-studied problem. NGINX [1] and HAProxy [2] are two widely-used production load balancers that support many algorithms. Kubernetes [3] has built-in load balancing through Services.

The classic algorithms include:

- **Round-robin**: cycles through workers in order – simple but blind to actual load [4].
- **Least-connections**: picks the worker with the fewest active requests – better under uneven loads [5].
- **Weighted strategies**: assign weights based on worker capacity or measured performance [6].
- **Latency-based**: use response-time measurements to prefer faster workers; Envoy Proxy [7] uses EWMA for this.

Our latency-aware algorithm follows Envoy's EWMA approach combined with a queue-depth penalty similar to [8]. The key difference in our work is that we study a heterogeneous scenario (one worker with 4× slowdown) and analyse how in-flight tracking must be done correctly – specifically, the in-flight counter must be incremented *before* any `await` yield point, and health-check latency must not pollute the EWMA used for routing.

III. System Design

A. Architecture Overview

Our system has four main components:

1) **Workers** (3 instances): FastAPI services that simulate analytics event processing. Each worker can introduce configurable blocking CPU work and asynchronous I/O delay.

2) **Load Balancer**: A custom FastAPI reverse proxy that intercepts all client requests and forwards them to a selected worker based on the active algorithm.
3) **Monitoring**: Prometheus scrapes metrics from the load balancer every 2 seconds. Grafana provides real-time dashboards.
4) **Load Generator**: k6 generates synthetic HTTP traffic with configurable virtual users, duration, and per-request parameters.

Everything runs in Docker containers orchestrated by Docker Compose on a single host.

B. Worker Service

Each worker exposes:

- `/api/v1/health` – health check endpoint
- `/api/v1/analytics/event` – processes an analytics event with configurable `delay_ms` (async sleep) and `cpu_ms` (synchronous busy-loop)
- `/api/v1/crash` – kills the process (for failure testing)

A `SLOWDOWN_FACTOR` environment variable scales both parameters. Worker-3 uses `SLOWDOWN_FACTOR=4`, so a request with `delay_ms=40` and `cpu_ms=5` triggers 160 ms of async sleep followed by 20 ms of synchronous CPU work – approximately 180 ms total, compared to 45 ms for the fast workers. The busy-loop runs on the Python asyncio event-loop thread, blocking all other coroutines for its duration.

C. Load Balancer Algorithms

1) Round-Robin (Baseline): Cycles through healthy workers in order. Uses a shared counter with an async lock. Workers marked unhealthy are skipped.

2) Least-Connections: Selects the worker with the fewest in-flight requests. Ties broken by EWMA latency. Adapts to slow workers because slow workers accumulate more in-flight connections and are thus naturally deprioritised.

3) Latency-Aware (EWMA + Queue Penalty): Computes a score for each worker:

$$\text{score}(w) = \text{ewma\_latency}(w) + 5 \times \text{in\_flight}(w) \quad (1)$$

The worker with the lowest score is selected. EWMA is updated after every completed request (not during health checks, which would pollute the routing signal):

$$\text{ewma}_{new} = \alpha \times \text{measured} + (1 - \alpha) \times \text{ewma}_{old}, \quad \alpha = 0.25 \quad (2)$$

An important implementation detail: the in-flight counter is incremented *before* reading the request body (an asyncio yield point), ensuring that concurrent coroutines see accurate in-flight counts when selecting a worker.

D. Health Checking

A background loop pings every worker via `GET /api/v1/health` every 1 s with an 800 ms timeout. If a worker fails to respond or returns 5xx, it is marked unhealthy and removed from the candidate pool. Recovery is automatic on the next successful check.

E. Metrics and Observability

The load balancer exposes Prometheus-format metrics at `/lb/metrics`, including total and failed request counts, p95/p99 latency over a rolling 1000-request window, and per-worker health, in-flight, EWMA latency, score, and request counts. A web-based control panel allows operators to trigger k6 load-test scenarios from the browser and stream their output in real time.

IV. Integrated Specification

Table I lists the system requirements and how each is verified.

TABLE I
System Requirements

ID	Requirement	Priority	Verification
R1	At least 3 workers	Must	Docker Compose
R2	Round-robin routing	Must	Functional test
R3	Least-connections routing	Must	Functional test
R4	Latency-aware routing	Must	Functional test
R5	Health checks	Must	HTTP test
R6	Prometheus metrics	Must	Metrics scrape
R7	Synthetic workload	Must	k6 experiment
R8	Failure injection	Must	Worker crash
R9	Live dashboard	Should	Browser test
R10	Heterogeneous workers	Should	Config review
R11	Reproducible results	Should	Script execution

V. Experiment Setup

A. Environment

All experiments run on a personal home server with an Intel Core i5-7500T CPU (2.70 GHz, 4 cores) and 15 GB RAM running Debian Linux with Docker Compose. Container-to-container network latency is sub-millisecond and negligible compared to request delays. Each scenario is run independently: the full stack is torn down and rebuilt between runs to ensure clean state.

B. Workload Parameters

Each request carries `delay_ms=40` and `cpu_ms=5`. For workers 1 and 2 this means 40 ms of async sleep followed by 5 ms of synchronous CPU work per request – approximately 45 ms total response time. For worker-3 (`SLOWDOWN_FACTOR=4`) this becomes 160 ms of async sleep followed by 20 ms of synchronous CPU blocking – approximately 180 ms total, a 4× slowdown. The load balancer's request timeout is 2000 ms; any request that does not complete in time returns a 502 error.

C. Scenarios

1) **Constant Load**: 30 VUs, 60 s. Baseline comparison across algorithms.
2) **Traffic Spike**: Ramps 5→80 VUs in 5 s, holds 30 s, ramps back.
3) **Stress Test**: Ramps from 5 to 300 VUs over 70 s to find the saturation point.

4) **Failure Injection**: Constant 30 VUs with the latency-aware algorithm; worker-2 is killed at $t = 20$ s.

VI. Results

A. Scenario 1: Constant Load

Table II shows results for 30 VUs over 60 s. All latency and throughput numbers come from k6's summary export; traffic distribution is from the load balancer's own worker counters.

TABLE II
Constant Load Results (30 VUs, 60 s)

Metric	Round-Robin	Least-Conn	Latency-Aware
Avg latency (ms)	107.1	79.7	74.4
p95 latency (ms)	224.9	220.1	182.4
Throughput (req/s)	143.9	165.7	170.7
Error rate (%)	0.0	0.0	0.0
Traffic to worker-3 (%)	33.3	13.4	4.9

Under constant moderate load, all three algorithms achieve zero errors, but their traffic distributions differ markedly. Round-robin routes exactly one-third of requests to each worker as expected. Least-connections reduces worker-3's share to 13.4%, because its higher response time (180 ms vs. 45 ms) means it naturally accumulates more in-flight requests and is deprioritised by the min-connections selector. Latency-aware is the most aggressive: it routes only 4.9% of traffic to worker-3, whose EWMA latency (∼200 ms) makes it score four times worse than the fast workers (∼49 ms).

The performance impact of this routing is clear: latency-aware achieves 74.4 ms average latency – 30% lower than round-robin's 107.1 ms – and 18.6% higher throughput (170.7 vs. 143.9 req/s). Least-connections falls in between, with 79.7 ms average latency and 15% higher throughput than round-robin.

B. Scenario 2: Traffic Spike

Table III shows spike-test results. The burst to 80 VUs pushes throughput up substantially for the adaptive algorithms while keeping errors at zero.

TABLE III
Spike Test Results (5 → 80 VUs)

Metric	Round-Robin	Least-Conn	Latency-Aware
Avg latency (ms)	392.1	182.0	172.2
p95 latency (ms)	1401.8	485.3	362.3
Throughput (req/s)	114.5	216.3	225.8
Error rate (%)	0.0	0.0	0.0
Traffic to worker-3 (%)	33.3	15.4	12.0

The spike scenario exposes the full cost of blind routing. Round-robin continues to send one-third of traffic to the slow worker, which cannot keep up at 80 VUs; this backs up the queue and drives average latency to 392 ms with a p95 of 1402 ms – likely due to queuing at the slow worker. Latency-aware and least-connections both limit worker-3 to 12–15% of traffic, sustaining average latencies of 172 ms and 182 ms respectively, and throughput nearly double that of round-robin (216–226 vs. 114 req/s). No errors occur because even the slow worker completes within the 2000 ms timeout at these concurrency levels.

C. Scenario 3: Stress Test

Table IV shows stress-test results, where 300 VUs push the system well past its comfortable operating range.

TABLE IV
Stress Test Results (5 → 300 VUs)

Metric	Round-Robin	Least-Conn	Latency-Aware
Avg latency (ms)	595.4	440.9	440.8
p95 latency (ms)	2524.7	2601.7	2276.5
Throughput (req/s)	173.3	231.7	228.7
Error rate (%)	7.59	1.63	0.80
Traffic to worker-3 (%)	33.3	16.8	12.7
Workers healthy at end	None	All	All

Under extreme stress the differences are stark. Round-robin saturates all three workers equally, marking all of them unhealthy by the end of the test, and produces 7.59% errors at only 173 req/s. Both adaptive algorithms maintain all workers healthy throughout, achieve over 228 req/s (32% higher than round-robin), and keep error rates at 0.8–1.6%.

The latency-aware algorithm has a slight edge over least-connections in error rate (0.80% vs. 1.63%) and p95 latency (2277 ms vs. 2602 ms). Both algorithms route only 13–17% of traffic to the slow worker, sparing the fast workers from the extra queuing that overwhelms round-robin.

D. Scenario 4: Failure Injection

The experiment used the latency-aware algorithm with 30 VUs for 60 s; worker-2 was killed at $t = 20$ s. Table V summarises the results.

TABLE V
Failure Injection Results (latency-aware, 30 VUs)

Metric	Value
Worker crashed	worker-2 at $t = 20$ s
Detection time	<1 s (health-check interval)
k6 avg latency (full test)	94.1 ms
k6 p95 latency (full test)	241.5 ms
k6 throughput	153.6 req/s
Overall error rate (LB stats)	0.01% (1 of 9252 requests)
Worker-1 requests (no failures)	6 462
Worker-2 requests (stopped after crash)	1 585
Worker-3 requests (no failures)	1 205

The failure injection experiment demonstrates near-perfect resilience. Worker-2 served 1585 requests before being crashed. After the crash, the health-check loop detected the failure within one second and stopped routing to worker-2. Only one request failed (a 502 proxy error) in the entire 9252-request test – an overall failure rate of 0.01%.

After the crash, worker-1 absorbed the bulk of the redirected traffic (reaching 6462 total requests), while worker-3 received

a secondary share (1205 requests). The latency-aware algorithm correctly identified worker-1 as the fastest remaining option (EWMA ∼67 ms vs. worker-3's ∼215 ms). The overall average latency of 94 ms and p95 of 242 ms show that the system continued operating normally throughout.

VII. Discussion

A. Key Findings

1) **Adaptive algorithms consistently outperform round-robin.** Across all three non-failure scenarios, both least-connections and latency-aware routing reduced average latency, increased throughput, and (under stress) dramatically cut failure rates. The benefit appears even under moderate load, not only at saturation.
2) **Latency-aware routing most aggressively avoids slow workers.** By combining EWMA latency with in-flight count in a single score, the latency-aware algorithm sends only 5–13% of traffic to the slow worker across all scenarios, compared to 13–17% for least-connections and 33% for round-robin. This translates directly into lower average latency and p95.
3) **Stress testing reveals correctness requirements.** At 300 VUs, round-robin drives all workers unhealthy; adaptive algorithms keep all workers healthy. The difference comes from the in-flight counter: adaptive algorithms avoid queuing too deeply at any single worker, preventing the timeout cascade that forces health checks to fail.
4) **Failure recovery is near-instantaneous.** The 1-second health-check interval is sufficient for practical crash detection. With the latency-aware algorithm, the system loses only one request during a complete worker crash, and re-balances traffic across the two survivors within seconds.
5) **Implementation correctness matters.** Two subtle bugs during development caused all algorithms to produce identical 33% distributions: (a) updating EWMA with health-check latency polluted the routing signal, and (b) incrementing the in-flight counter after an asyncio yield point allowed concurrent coroutines to see stale zero counts. Both bugs had to be fixed for the algorithms to behave as intended.

B. Trade-offs

- **Throughput vs. tail latency under stress:** All algorithms exhibit high p95 values at 300 VUs (>2000 ms) because the system is overloaded. The adaptive algorithms maintain lower *average* latency and error rates because they avoid saturating the slow worker, but cannot eliminate tail latency when fast workers are also overloaded.
- **Overhead:** Round-robin is O(1); least-connections and latency-aware are O(n) over healthy workers. With 3 workers this is negligible; at hundreds of backends it would need optimisation (e.g., a min-heap).
- **EWMA convergence:** With $\alpha = 0.25$, the EWMA takes roughly 10–15 requests to converge from the initial 100 ms to the true value. During this warmup, routing is less informed. Shorter-lived tests or frequent restarts would penalise latency-aware more than the other algorithms.

C. Limitations

- **Single-host deployment.** All containers share the same 4-core CPU and single network interface. At high VU counts, CPU contention between k6, the load balancer, and the workers may affect results. On a real distributed system, network latency and independent CPU pools would change the picture.
- **Fixed EWMA α.** The value 0.25 was chosen empirically; a systematic sweep would be needed to characterise the stability/responsiveness trade-off.
- **Rolling latency window.** The load balancer's p95/p99 statistics cover only the last 1000 requests, so values captured at the end of a spike or stress run reflect the final load level, not the worst point during the test.
- **Short experiments.** At 60–70 s per run, long-term stability patterns and EWMA re-convergence after transient events cannot be studied.
- **No retry logic.** Failed requests are not retried. A production load balancer would combine routing with a retry budget to further reduce client-visible failures.

D. Safety, Privacy, and Deployment Notes

- The crash endpoint (`/api/v1/crash`) and management endpoints (`/lb/stats`, `/lb/workers`) expose internal system state and must be access-controlled in production.
- The system assumes fail-stop workers. Byzantine failures (wrong data returned) are not handled.
- All experiments are scripted and containerised; results are reproducible on any Docker host, though absolute latency numbers will vary with hardware.
- This is a course prototype. A production load balancer additionally requires TLS termination, connection pooling, circuit breakers, and retry budgets.

VIII. Conclusion

We designed and evaluated a containerised load-balancing testbed comparing round-robin, least-connections, and latency-aware routing under four workload scenarios. The central finding is that adaptive algorithms provide measurable and substantial benefits in heterogeneous deployments, even under moderate load. Latency-aware routing reduces average latency by 30% and increases throughput by 19% at constant moderate load (30 VUs) compared to round-robin. Under extreme stress (300 VUs), both adaptive algorithms sustain 32% higher throughput with 5–10× lower failure rates and keep all workers healthy, while round-robin saturates every worker and drives failure rates above 7%. In the failure-injection experiment, the latency-aware algorithm recovers

from a complete worker crash with a single failed request out of 9252 (0.01% failure rate).

A secondary finding is that implementation correctness is essential: two subtle bugs (health-check latency polluting the EWMA, and in-flight counter incremented after an asyncio yield point) caused all algorithms to behave identically. Fixing these two bugs was necessary for the algorithms to exhibit their intended differentiated behaviour.

Future work could explore shorter health-check intervals, circuit-breaker patterns, thread-pool isolation for CPU-bound workers, reinforcement-learning-based routing under non-stationary workloads, and the effect of varying the EWMA decay factor α.

REFERENCES

[1] NGINX Inc., "NGINX Load Balancing," https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/, accessed 2024.

[2] HAProxy Technologies, "HAProxy Configuration Manual," https://docs.haproxy.org/, accessed 2024.

[3] The Kubernetes Authors, "Service – Kubernetes Documentation," https://kubernetes.io/docs/concepts/services-networking/service/, accessed 2024.

[4] A. S. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed. Pearson, 2017.

[5] V. Cardellini, M. Colajanni, and P. S. Yu, "Dynamic load balancing on Web-server systems," *IEEE Internet Computing*, vol. 3, no. 3, pp. 28–39, 1999.

[6] V. S. Pai, M. Aron, G. Banga, M. Svendsen, P. Druschel, W. Zwaenepoel, and E. Nahum, "Locality-aware request distribution in cluster-based network servers," in *Proc. ASPLOS*, 1998, pp. 205–216.

[7] Envoy Proxy, "Load Balancing Policies," https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/load_balancing/load_balancers, accessed 2024.

[8] M. Mitzenmacher, "The power of two choices in randomized load balancing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 12, no. 10, pp. 1094–1104, 2001.